SPACEWISE AI

Informe técnico de avance

Simulación y comparación del consumo energético de maquinaria industrial y de centros de investigación

Programa Regional Siemens 2026, «Potencia tus conocimientos en Eficiencia Energética».
Grupo 3: Vicente Dauvergne, Rodrigo Vidal, Javier Riquelme y Matías Seguel. Tutores: Iván Ramírez y Thamara Villegas.
12 de octubre de 2026. Entrega del concurso: 23 de octubre.

1. Resumen

SPACEWISE AI nació como un gemelo digital del Centro Espacial Nacional de Chile (CEN). Al no obtener mediciones reales de esa instalación, el proyecto se reorientó a una plataforma general de simulación y comparación del consumo eléctrico de maquinaria, aplicable a la industria y a centros de investigación.

La versión actual del repositorio ya incluye un catálogo de 110 equipos, un motor de simulación en MATLAB que genera el perfil horario de cada máquina durante un año (8.760 horas), una base de datos SQLite, una API en Flask, un dashboard web y un orquestador en Python que lanza todo con un solo comando. La simulación puede correr de forma continua, hora por hora, y el dashboard se actualiza mientras avanza.

En esta revisión se probó la parte Python (39 pruebas aprobadas) y se levantó el conjunto orquestador, servidor y dashboard con un reemplazo de MATLAB. Se corrigieron varios errores que impedían ver el dashboard. El código MATLAB no se pudo ejecutar en el equipo de revisión, por lo que sus resultados numéricos todavía no están verificados. Todo el proyecto trabaja con datos de catálogo, no con mediciones reales.

2. Cambio de enfoque

El 11 de octubre el equipo decidió dejar el caso del CEN por la falta de respuesta de la institución. Lo que cambia:

	Versión inicial	Versión actual
Caso de estudio	Edificio del CEN	Maquinaria y equipos de industria y de centros de investigación
Unidad de modelado	Subsistemas repartidos entre 22 dispositivos	Cada máquina se simula por separado
Tamaño de la base	22 dispositivos	110 equipos, ampliable
Datos de entrada	Parámetros del CEN (supuestos)	Datos de placa y perfiles de uso del catálogo
Entrega	Gemelo, control y dashboard	Simulación y dashboard comparador

El modelo del edificio queda como antecedente en docs/archive y en los módulos dt_* del código; no se usa en el nuevo enfoque.

Quedan ideas por evaluar, como generación propia con turbinas pequeñas o sensores en zonas de consumo sin control. Todavía no forman parte del alcance.

3. Estado actual del proyecto

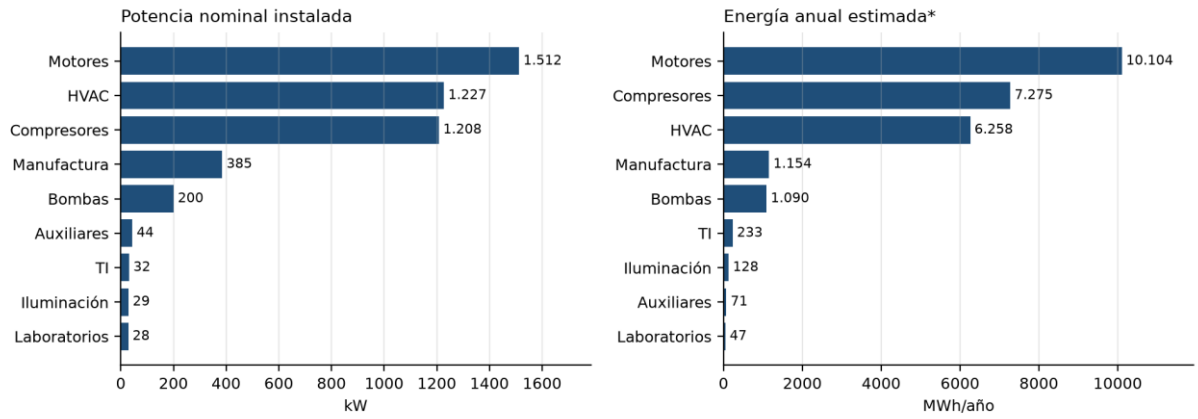
Componente	Qué hay en el repositorio	Estado
Catálogo de equipos	110 máquinas en docs/CATALOGO_EQUIPOS.md, con especificaciones, ciclos de trabajo y estacionalidad	Listo
Motor de simulación	Módulos sim_catalog, sim_machine, sim_engine, sim_analysis, sim_db y sim_export en MATLAB	Código listo, sin ejecutar en esta revisión
Simulación continua	Una hora simulada cada 2 s (configurable); retoma desde la última hora guardada	Código listo, sin ejecutar
Base de datos	SQLite con 8 tablas: máquinas, simulaciones, mediciones, totales horarios, resultados y datos importados	Listo
API	Flask con endpoints de máquinas, simulaciones, totales horarios, mediciones y exportación CSV/JSON	Probada (Python)
Dashboard	Panel web con Chart.js servido por Flask en la raíz	Probado con datos de prueba
Orquestador	orquestador.py: crea la base, lanza MATLAB y el servidor, abre el navegador	Probado con un MATLAB simulado
Pruebas	39 en Python (aprobadas) y 59 métodos de prueba en MATLAB (sin ejecutar)	Parcial
Comparación con datos reales	Solo existe el esquema de tablas (import_data); no hay código de importación	Pendiente
Predicción de demanda	No está en el nuevo simulador	Pendiente

4. Catálogo de equipos

El catálogo reúne 110 equipos en nueve categorías. Para cada uno define potencia nominal, tensión, fases, factor de potencia, horas y días de operación, factor de carga promedio, ciclos de trabajo, variabilidad estacional, tiempo y múltiplo de arranque, consumo en espera y criticidad (48 críticos, 40 esenciales, 13 flexibles y 9 no críticos).

Categoría	Equipos	Potencia nominal (kW)	Energía estimada* (MWh/año)
Motores eléctricos	20	1.512	10.104
HVAC industrial	20	1.227	6.258
Compresores y neumática	15	1.208	7.275
Equipos de manufactura	10	385	1.154
Bombas e hidráulica	15	200	1.090
Equipos auxiliares	5	44	71
Servidores y TI	5	32	233
Iluminación industrial	10	29	128
Laboratorios	10	28	47
Total	**110**	**4.666**	**26.360**

** Estimación simple: potencia nominal × factor de carga × horas por día × días por semana × 52,14 semanas. No es un resultado del simulador, que además considera ciclos, estacionalidad, arranques y espera.*



Potencia instalada y energía anual estimada por categoría (datos del catálogo)

Los motores, el HVAC y los compresores concentran el 85 % de la potencia instalada y cerca del 90 % de la energía estimada, por lo que ahí estarán las principales oportunidades de ahorro.

Importante: el propio catálogo indica que su fuente es un informe generado con Gemini Deep Research a partir de «Datos Equipos.pdf». Los fabricantes, modelos y valores deben contrastarse con fichas técnicas reales antes de presentarlos como datos de placa.

5. Cómo funciona la simulación

Para cada máquina, el motor genera 8.760 puntos horarios (un año a partir del 1 de enero de 2026). En cada hora decide primero si el equipo opera, según sus horas por día y días por semana. Si opera, determina su fase del ciclo de trabajo (plena carga, carga parcial, espera o arranque) con las fracciones de tiempo del catálogo, aplica un multiplicador estacional según la variabilidad indicada (alta, moderada o baja) y suma un ruido

aleatorio de 5 %. La potencia resultante es la nominal por el factor de carga; en espera se usa el porcentaje de espera, y en arranque, el múltiplo de arranque. La semilla de cada máquina se deriva de una semilla global más su identificador, así que los resultados se pueden repetir.

Con los perfiles de las 110 máquinas se calculan los totales por hora y por categoría. De ahí salen los indicadores que guarda la base: energía total, potencia promedio y máxima con su fecha, energía por categoría y por criticidad, las diez máquinas de mayor consumo, patrones mensual, semanal y diario, potencial de flexibilidad (energía de equipos flexibles), y factor de potencia y distorsión armónica promedio. Cada máquina tiene además sus propios resultados.

En modo continuo el motor genera todos los perfiles al inicio y luego escribe en la base una hora simulada cada pocos segundos. Un año completo tarda unas 4,9 horas con el intervalo de 2 s. Si se interrumpe, retoma desde la última hora almacenada.

6. Dashboard y orquestador

El dashboard muestra la energía total, la potencia promedio y máxima, la cantidad de máquinas, el consumo horario, el consumo por categoría y las diez máquinas con más consumo. Permite elegir la simulación y exportar los datos a CSV con un rango de fechas. Mientras hay una simulación en ejecución se actualiza solo cada 5 segundos.

El comando «python orquestador.py» deja todo funcionando: crea la base de datos con su esquema si no existe, inicia la simulación continua en MATLAB (con hasta tres reinicios automáticos si falla), inicia el servidor Flask, espera a que responda y abre el navegador en http://127.0.0.1:5000. Con Ctrl+C se detiene todo. Los registros de cada proceso quedan en data/logs. También existe el modo «once», que corre un año completo de una vez, y las opciones --interval y --no-browser.

7. Revisión técnica

Al revisar los últimos cambios encontramos problemas que explicaban por qué el dashboard no aparecía. Los corregidos se probaron con las pruebas de Python y una ejecución completa del orquestador con un MATLAB simulado.

N°	Problema	Estado
1	Flask no servía el dashboard: http://localhost:5000 respondía 404.	Corregido: ahora sirve index.html en la raíz
2	La simulación continua nunca se veía: el dashboard listaba solo simulaciones completadas y el resumen daba 404 porque ese modo no guarda resultados finales.	Corregido: muestra las que están en ejecución y calcula el resumen parcial desde los totales horarios
3	La tabla de máquinas quedaba vacía: ningún script cargaba el catálogo en la base. Sin ella fallaban el selector, el top 10 y los análisis por criticidad.	Corregido en MATLAB (ensureMachines); falta probarlo con MATLAB
4	Flask podía crear un archivo de base vacío antes de que MATLAB creara el esquema, y MATLAB no lo volvía a crear.	Corregido: conexión de solo lectura y esquema creado por el orquestador
5	El orquestador fallaba si MATLAB no estaba en el PATH (la ruta por defecto tenía un error de tipo), dependía de python-dotenv, ignoraba --interval, podía bloquearse por salidas sin leer y cerraba el servidor cuando MATLAB terminaba.	Corregido
6	El archivo .env con la ruta personal de MATLAB de un integrante estaba versionado, y Flask no figuraba en requirements.txt.	Corregido: .env fuera del repositorio y dependencias completas
7	El gráfico del top 10 leía campos distintos de los que guarda MATLAB.	Corregido: acepta ambos formatos
8	Dos pruebas fallaban en Windows (codificación de texto y mensaje de load_catalog).	Corregido
9	El selector de máquina y las fechas del dashboard no filtran los gráficos; solo afectan la exportación.	Pendiente
10	El catálogo es un solo archivo de texto; agregar un equipo implica editarlo. No hay una carpeta por máquina como se planteó.	Pendiente, según el diseño que se defina
11	No se ejecutó MATLAB en esta revisión: los cambios en sim_db, sim_engine y run_continuous no se probaron.	Pendiente: correr runtests y una simulación

8. Pendientes

- Ejecutar en una máquina con MATLAB las pruebas del simulador y una corrida completa, y revisar los totales anuales contra la estimación del catálogo.
- Contrastar los datos de los 110 equipos con fichas técnicas de fabricantes.
- Definir cómo se comparan equipos y escenarios en el dashboard, que es el objetivo central del nuevo enfoque (por ejemplo, un equipo estándar contra uno de alta eficiencia, o con y sin variador).
- Incorporar la predicción de demanda y la importación de datos reales para comparar contra la simulación.
- Hacer que los filtros de máquina y fecha afecten a los gráficos.
- Decidir si el catálogo pasa a una estructura de carpetas por máquina.

Anexo. Cómo ejecutar

- Requisitos: MATLAB R2026a o superior con Database Toolbox y Python 3.10 o superior. Instalar las dependencias con «pip install -r requirements.txt».
- Si MATLAB no está en el PATH, definir la variable MATLAB_BIN o copiar .env.example a .env y fijar DEFAULT_MATLAB.
- Ejecutar «python orquestador.py». El dashboard queda en http://127.0.0.1:5000.
- Pruebas de Python: «pytest web/backend/tests tests». Las pruebas de MATLAB y de integración requieren MATLAB.